

Automata Theory and Compiler Design Mini Project

PDA for Language $L = \{ w C w^R \mid w \in \{0,1\}^* \}$

Department of Computer Science & Engineering
National Institute of Technology, Patna

Submitted By:

Name: _____ Roll No.: _____

Name: _____ Roll No.: _____

Name: _____ Roll No.: _____

Course Code : _____

Course Title: _____

Branch : Computer Science and Engineering

Table of Contents

1. Aim of the Experiment
2. Introduction
 - 2.1 Background
 - 2.2 Problem Statement
 - 2.3 Objective
 - 2.4 Scope and Limitations
3. Theoretical Background
 - 3.1 Pushdown Automata (PDA)
 - 3.2 Language $L = \{ w C w^R \mid w \in \{0,1\}^* \}$
 - 3.3 Informal Working of the PDA
4. Requirements
 - 4.1 Hardware Requirements
 - 4.2 Software Requirements
5. Design
 - 5.1 System Overview
 - 5.2 Transition Description
 - 5.3 Algorithm
6. Implementation (C Code)
7. Sample Input and Output
8. Observations
9. Conclusion
10. Learning Outcomes
11. References

1. Aim of the Experiment

The aim of this mini project is to design and implement a pushdown automaton (PDA) that accepts the language $L = \{ w C w^R \mid w \text{ is any binary string over } \{0,1\} \text{ and } w^R \text{ is its reverse, } C \text{ is a special separator symbol} \}$. A C program is used to simulate the behavior of this PDA.

2. Introduction

2.1 Background

Pushdown automata extend finite automata with an auxiliary stack memory, which allows them to recognize context-free languages. Many non-regular languages, such as palindromes and matched parentheses, can be modeled naturally using PDAs by pushing input symbols onto the stack and later popping and matching them.

2.2 Problem Statement

Design a PDA that recognizes strings of the form $w C w^R$, where w is any binary string over $\{0,1\}$, w^R is the reverse of w , and C is a special symbol that does not appear inside w . The PDA should first read and store the prefix w using its stack, then, after reading C , compare the remaining input with the stack contents in reverse order.

2.3 Objective

The objectives of this project are:

- To understand the working of pushdown automata for non-regular languages.
- To construct a PDA for the language $L = \{ w C w^R \mid w \in \{0,1\}^* \}$.
- To implement a simulator in C that checks whether a given input string belongs to this language.
- To reinforce the connection between formal automata theory and practical implementation.

2.4 Scope and Limitations

The project focuses on a deterministic PDA that accepts binary strings of the form $w C w^R$. The implementation assumes exactly one occurrence of the separator symbol C in the input and does not attempt to recover from malformed inputs with multiple separators or invalid characters. The simulator operates on finite-length input strings provided by the user.

3. Theoretical Background

3.1 Pushdown Automata (PDA)

A pushdown automaton is defined as a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where Q is the set of states, Σ is the input alphabet, Γ is the stack alphabet, δ is the transition function, q_0 is the start state, Z_0 is the initial stack symbol, and F is the set of accepting states. Transitions can read an input symbol (or ϵ), examine the top of the stack, and replace it with another stack string. This additional stack memory makes PDAs strictly more powerful than finite automata.

3.2 Language $L = \{ w C w^R \mid w \in \{0,1\}^* \}$

The language considered here consists of strings over the alphabet $\{0,1,C\}$ where the portion before C is an arbitrary binary string w and the portion after C is exactly the reverse of w . For example, $0C0$, $10C01$, and $101C101$ are in the language, whereas $10C10$ or $11C11$ are not. The separator C uniquely identifies the midpoint of the string.

3.3 Informal Working of the PDA

Informally, the PDA operates in two phases. In the first phase, while reading the prefix w from left to right, every 0 or 1 is pushed onto the stack. When the symbol C is encountered, the PDA switches to the second phase. In this phase, for each input symbol in the suffix, it pops and compares with the top of the stack. If every symbol matches and the stack is empty exactly when the input ends, the string is accepted.

4. Requirements

4.1 Hardware Requirements

- A computer system with at least:
 - Processor: Intel i3 or equivalent
 - RAM: 4 GB or more
 - Storage: 500 MB free disk space
- Any modern operating system capable of compiling and running C programs.

4.2 Software Requirements

- Operating System: Windows / Linux / macOS
- C Compiler: GCC / Clang / MSVC or any standard-compliant C compiler
- Text editor or IDE for writing and compiling C code (e.g., VS Code, Code::Blocks, Vim).

5. Design

5.1 System Overview

The PDA is simulated by a C program that reads an input string containing the characters 0, 1, and C. While scanning from left to right, the program uses an array to implement the stack. Before encountering C, all 0s and 1s are pushed. After C, each new symbol must match the top stack symbol, which is then popped. If the string is processed completely and the stack is empty, the input is accepted.

5.2 Transition Description

The PDA can be described using three main states: q0 (pushing phase), q1 (matching phase), and q_accept (accepting). In state q0, on reading 0 or 1, the symbol is pushed onto the stack and the automaton stays in q0. On reading C, the PDA switches to state q1 without modifying the stack. In state q1, for each input symbol 0 or 1, the PDA pops and compares the top of the stack; if they match, it continues, otherwise it rejects. When the input is finished and the stack contains only the initial marker, it moves to q_accept.

5.3 Algorithm

High-level algorithm for the simulator:

1. Initialize an empty stack.
2. Scan the input from left to right until C is found, pushing each 0 or 1 onto the stack.
3. If C is missing or appears more than once, reject.
4. After C, for each remaining input symbol:
 - a. If the stack is empty, reject.
 - b. Pop the top of the stack and compare; if it differs, reject.
5. After the input ends, if and only if the stack is empty, accept; otherwise reject.

6. Implementation (C Code)

Below is the C implementation that simulates the PDA for the language $w C w^R$.

```
#include <stdio.h>
#include <string.h>

#define MAX_LEN 256

int main() {
    char input[MAX_LEN];
    char stack[MAX_LEN];
    int top = -1;
    int i = 0;
    int len;
    int c_pos = -1;

    printf("Enter a string over {0,1,C}: ");
    if (!fgets(input, sizeof(input), stdin)) {
        return 1;
    }

    // Remove trailing newline if present
    len = strlen(input);
    if (len > 0 && input[len - 1] == '\n') {
        input[len - 1] = '\0';
        len--;
    }

    // Find position of separator C and ensure it appears exactly once
    for (i = 0; i < len; ++i) {
        if (input[i] == 'C') {
            if (c_pos != -1) {
                printf("Rejected: more than one C found.\n");
                return 0;
            }
            c_pos = i;
        }
    }

    if (c_pos == -1) {
        printf("Rejected: no separator C found.\n");
        return 0;
    }

    // Phase 1: push all symbols of w onto the stack
    for (i = 0; i < c_pos; ++i) {
        if (input[i] != '0' && input[i] != '1') {
            printf("Rejected: invalid symbol before C.\n");
            return 0;
        }
        stack[++top] = input[i];
    }

    // Phase 2: compare with reverse using the stack
    for (i = c_pos + 1; i < len; ++i) {
        if (input[i] != '0' && input[i] != '1') {
            printf("Rejected: invalid symbol after C.\n");
            return 0;
        }
    }
    if (top < 0) {
        printf("Rejected: more symbols after C than before.\n");
        return 0;
    }
    char top_sym = stack[top--];
    if (top_sym != input[i]) {
        printf("Rejected: mismatch between stack and input.\n");
    }
}
```

```
        return 0;
    }

    if (top == -1) {
        printf("Accepted: string is of the form w C w^R.\n");
    } else {
        printf("Rejected: fewer symbols after C than before.\n");
    }

    return 0;
}
```


7. Sample Input and Output

Example console interactions with the PDA simulator are shown below:

```
Enter a string over {0,1,C}: 10C01
Accepted: string is of the form  $w C w^R$ .
```

```
Enter a string over {0,1,C}: 10C10
Rejected: mismatch between stack and input.
```

```
Enter a string over {0,1,C}: 0C0
Accepted: string is of the form  $w C w^R$ .
```

8. Observations

- The PDA successfully accepts strings where the segment after C is the reverse of the segment before C.
- Incorrect placement of C or mismatched symbols causes early rejection, illustrating the role of the stack.
- The implementation closely follows the theoretical PDA design using a simple array-based stack.
- The approach can be generalized to other context-free languages that require matching of symmetric parts.

9. Conclusion

This mini project demonstrates how pushdown automata can recognize non-regular languages by exploiting stack memory. The constructed PDA for $L = \{ w C w^R \mid w \in \{0,1\}^* \}$ was translated into a working C program that simulates its behavior. Through this implementation, the connection between formal models of computation and practical programming was reinforced.

10. Learning Outcomes

Through this experiment, the following concepts and skills were learned:

- Formal definition and operation of pushdown automata.
- Design of a PDA for a specific context-free language.
- Implementation of stack-based automata behavior in the C programming language.
- Strengthening the link between automata theory and compiler design foundations.

11. References

1. John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, "Introduction to Automata Theory, Languages, and Computation", Pearson.
2. Any standard textbook on Compiler Design and formal languages.
3. Online resources discussing PDAs and context-free languages.